TrailTrack

Real-Time Trail Conditions and Community Walks for Ireland

Iteration 1 Report: Release v0.1

Mark Mukiiza

C00166672

BSc (Hons) Software Development, Level 8

South East Technological University, Carlow

October 2026

1. What Was Planned

The specification planned five features for Iteration 1 (weeks 1-6), all for the trails in County Laois:

- Import trail geometry from the Sport Ireland National Trails Register [1, 2] into PostgreSQL with PostGIS.
- User registration with an email and password, stored on the server.
- A map showing the imported trails over an OpenStreetMap base layer [3].
- Submitting a condition report against a trail, online only: the trail, a condition type and an optional note.
- Basic trail search by name, or by distance from the user's location.

Offline sync, moderation rules, community walks, ratings and advanced search were out of scope. The work was planned as 14 issues on the project board (Figure 2) for release v0.1, due on 30 October. Nine were features, including three screens from the wireframes: trail details, a location button and bottom navigation. Five were release tasks: code documentation, the release itself, a screencast, this report and a poster.

2. What Work Was Completed

All 14 issues were completed and merged through 8 pull requests into release v0.1. The app is built with Flutter, and the backend with Spring Boot, PostgreSQL and PostGIS. The register is published under CC BY 4.0 [4] and OpenStreetMap under the ODbL [5], and the app credits both.

Feature	What v0.1 does
Trail import (#2)	An administrator imports the Laois trails from the register. Running it again updates them instead of creating duplicates.
Map	Routes in the colour of their trail signs, grouped into sites, with a chooser where routes share a path (Figure 1).
Accounts (#3, #4)	Registration and login with an email and password. Logins last 7 days and survive the app being closed.
Trail details (#5)	Facts, current conditions and the reports from the last 7 days.
Condition reports (#6, #7)	Logged-in users report one of 9 condition types with a severity and an optional note. Anyone can read reports.
Search and location (#8, #9)	By name, or near the user within 5, 10 or 25 km. A button centres the map on the user.
Navigation (#10)	Map, Search and Profile tabs.
Release (#11-#15)	Code documentation, the tagged release, a screencast, this report and a poster.

Table 1: Work completed in Iteration 1

Part	Tests	What they cover
Backend	70	Unit tests of services, web layer tests including the 401 and 403 security responses, and integration tests against PostgreSQL with PostGIS, rolled back after each test.
App	101	Unit tests of models, API clients and view models with fake services, and widget tests of each screen on a phone-sized display.

Table 2: Automated tests in v0.1

I asked Claude Code to write the tests with each feature, not afterwards, following conventions I set, such as naming each backend test class after the class it tests and describing each test in plain English. I read the tests to check that they test real behaviour, and before every pull request I ran both suites and checked the feature on the Android emulator against the real backend. The backend tests use JUnit 5 and Mockito. The integration tests caught a real bug: the reference that stops a report being stored twice was never saved.

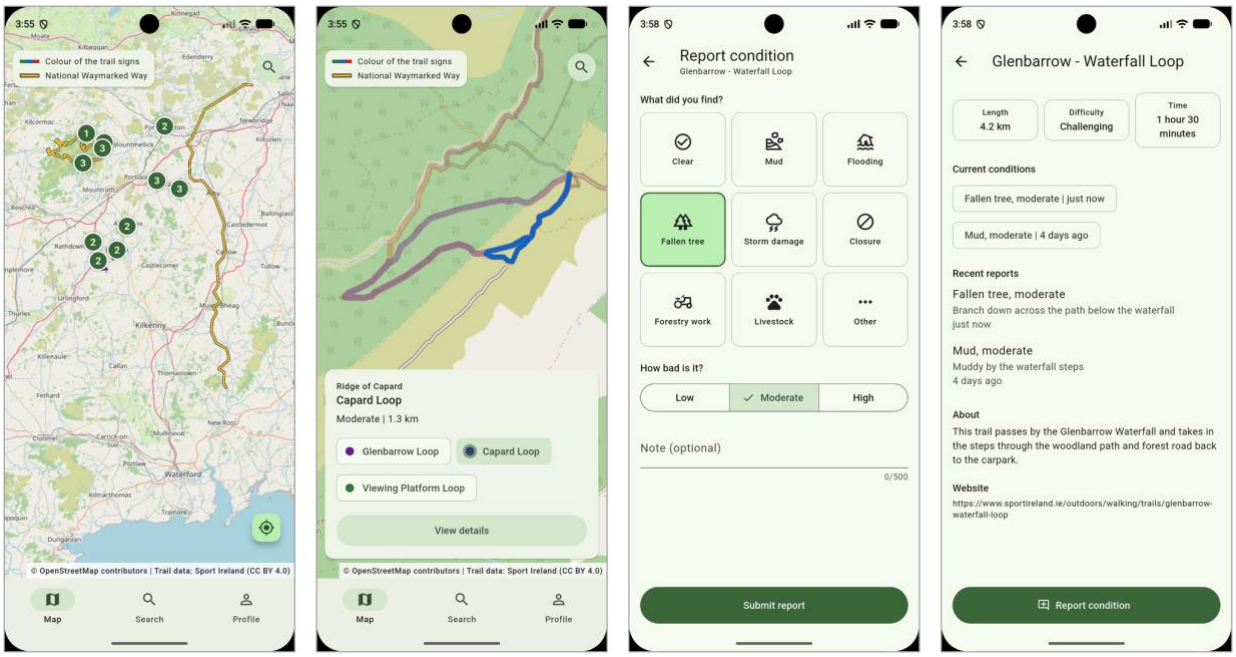


Figure 1: The map, a selected route, the report form, and a trail with a new report

3. What Worked

- **Importing the register once.** Trails are imported into PostgreSQL by an administrator, not fetched from the register on every request. The app keeps working if the register is slow or changes its format.
- **PostGIS for the spatial work.** It simplifies each route for the map (about 78 KB for all 28 trails), groups trailheads into sites, and finds trails within a distance using a spatial index.
- **Integration tests against the real database.** They found bugs that unit tests with mocks could not. For example, the reference that stops a report being stored twice was never saved, and only a test against PostgreSQL showed it.
- **Checking each feature on the emulator.** Running against the real backend found layout bugs that tests did not, such as the map fitting to the whole world and the system gesture bar covering the map credits.
- **Planning in GitHub.** Each release has a milestone, each feature an issue, and each feature a branch and pull request. The board shows what is left at a glance (Figure 2).

4. What Did Not Work

- **A temporary account button.** The login screens were built before the bottom navigation shown in the wireframes. A small account button on the map stood in for it, then was removed when the navigation was built.
- **The database schema is generated.** Hibernate creates the tables, but it cannot create the spatial index the distance search needs, so a class creates that index at startup. Database migrations are planned for the start of Iteration 2.
- **Location on the emulator.** The usual command for setting the emulator's position did not change Android's location, so "Near me" could only be tested once a test location provider was used.
- **Documentation tools.** Doxygen does not support Dart, so the app is documented with dart doc. Doxygen also skips Java records, so a small filter presents them to it as classes.

5. Problems Encountered

Table 3 lists the main problems found during the iteration and how each was solved.

Problem	Cause	Solution
Cross-county trails were missing.	The County field lists every county a trail crosses, e.g. "Laois, Offaly". An exact match found 24 of the 28 trails and dropped the Slieve Bloom Way and the Barrow Way.	Filter with County LIKE '%Laois%'.
Coordinates were not latitude and longitude.	The register returns Irish Grid coordinates (EPSG:29903) by default.	Request WGS84 (outSR=4326), which PostGIS and the map library expect.
Routes had no start point.	The register's Latitude and Longitude fields are empty for every Laois route.	Take the official start from the register's trailheads layer, matched by trail ID.
The map zoomed out to the whole world.	The camera was fitted to the trails before the map had a size.	Fit the camera only once the map reports that it is ready.
Routes from one car park overlapped.	Several loops start at the same trailhead and share paths.	Group trailheads within 10 m of each other into sites with PostGIS (28 routes in 13 sites). Each site has a pin, and chips switch between its routes.
Shared paths hid each other.	Routes on the same path were drawn on top of each other.	Draw longer routes first and wider, so shared paths show nested colour bands. A tap on a shared path asks which route was meant.
A report could be stored twice.	A report sent again after a dropped connection looked like a new one.	The app gives each report a unique reference. The server returns the stored report when it sees the same reference again.
Google asked to collect location data.	The location plugin used Google Play services by default, which shows a data-sharing prompt.	Use Android's own location manager instead.
"Near me" sometimes searched by name only.	Choosing a radius before the position was found ran a search without it.	Wait for the position before searching. Found while recording the screencast.

Table 3: Problems found in Iteration 1 and their solutions

5.1 How the Register's Fields Were Found

The register is published as two ArcGIS feature services: trail routes and trailheads [1, 2]. Neither has written documentation of its fields, so they were found in two steps.

- **Layer metadata.** Adding ?f=json to a layer's address [6] returns its field names and types, its geometry type and its coordinate system. This showed the fields to import and that coordinates default to the Irish Grid.
- **Test queries in Postman.** Each assumption was checked before the import relied on it. Counting with returnCountOnly compared the exact and LIKE county filters (24 against 28). Querying fields without geometry showed that Latitude and Longitude are empty. Listing the trailhead types showed Start, Finish and Secondary, of which only Start marks a route's official start.

The requests are saved as a Postman collection, so the checks can be repeated if the register changes. The same approach ruled out another data source: Coillte's felling licence layer is an archive of applications up to 2025, not a live list of closures [7].

6. Changes Made

6.1 From Difficulty Colours to Waymark Colours

The first version of the map coloured each route by difficulty: easy routes in solid green, moderate in dashed orange and challenging in dotted red. It looked clear, but it did not match what walkers see on the grounds.

Official trail maps, such as Coillte's maps for Oughaval Woods, Glenbarrow and Monicknew [8], identify each route by the colour of its waymark arrows, never by difficulty. Where routes share a path, they draw the coloured lines side by side. A walker at a trailhead follows a colour, so the app should use the same colours.

The register describes each route's signs in a text field, for example "Blue arrow on white background". The import turns this into a colour: the arrow's colour, or the background's when the arrow is white. A yellow arrow on black marks a National Waymarked Way, drawn in yellow with a black edge. Four routes have no usable colour and get neutral colours instead. Difficulty and length moved into the route card (Figure 1).

6.2 Other Changes

- **Routes grouped into sites.** The specification did not plan for this. Routes that start at the same car park share one pin, with chips to switch between them, so the map stays readable when zoomed out.
- **The import is for administrators only.** The import endpoint was restricted to administrator accounts, and the first administrator is created from environment variables when the backend starts.
- **Reporting needs an account.** Guests can read every report, but only logged-in users can post one, so others can trust where reports come from.
- **Three screens from the wireframes were added:** trail details, the location button and bottom navigation, which replaced the temporary account button.
- **Moved to later iterations:** database migrations with Flyway (Iteration 2) and email verification (Iteration 3).

7. Plan for Iteration 2

Iteration 2 runs to release v0.2, due on 4 December. Its main feature is community walks, the project's main difference from existing general-purpose hiking platforms. Table 4 lists the planned work; each item will be broken into smaller issues at the start of the iteration.

Issue	Work
#21	Database migrations with Flyway, replacing the generated schema and the startup index. Refine the data model, including when reports expire.
#16	Community walks: create a walk on a trail, with a date, meeting point and attendee limit.
#17	Community walks: browse upcoming walks and RSVP. Attendee lists are visible only to attendees and the organiser.
#18	Live condition overlays on the map, so a trail's current state shows without opening it.
#19	An optional hazard location on reports, snapped to the trail and shown as a pin, never with the reporter.
#20	Filter trail search by difficulty.

Table 4: Planned work for Iteration 2

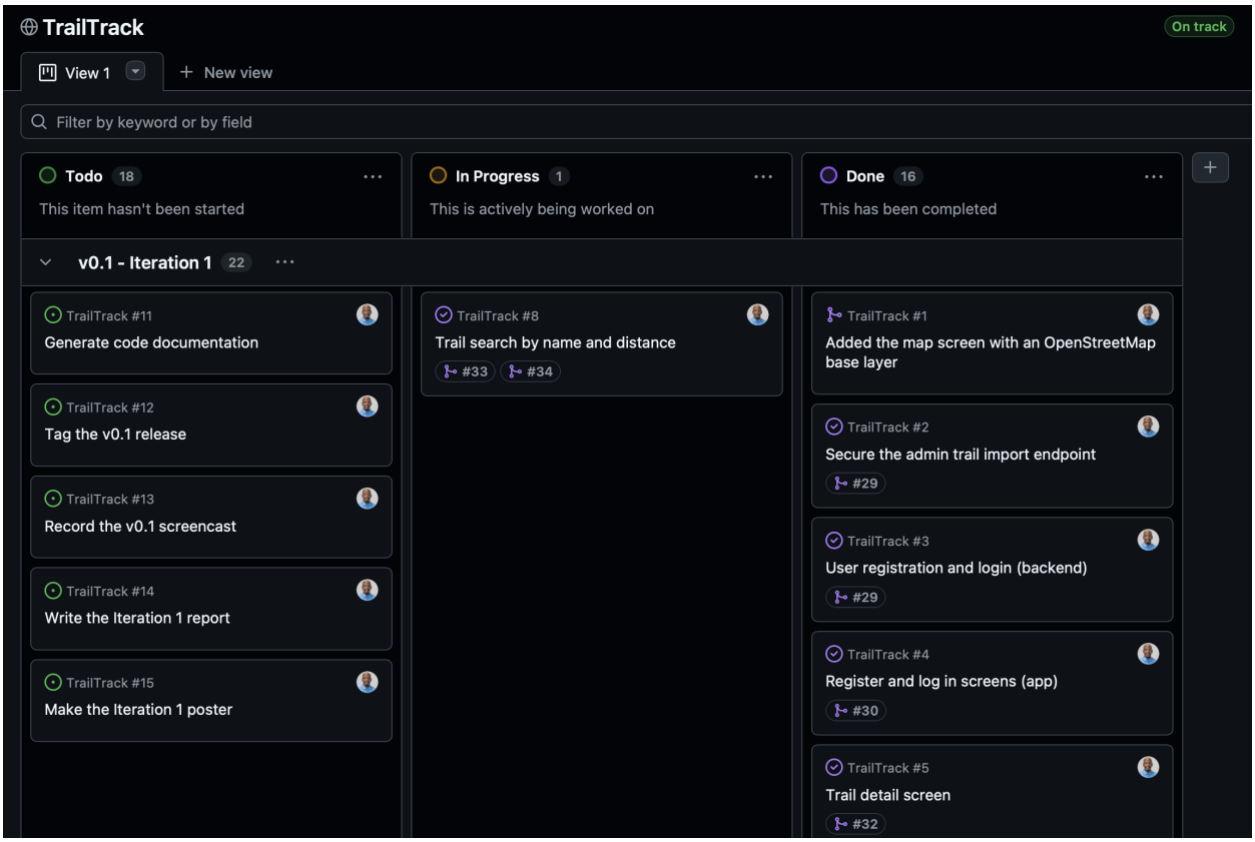


Figure 2: The TrailTrack project board

8. AI Usage

I used Claude Code, Anthropic's AI coding assistant [9], throughout Iteration 1.

8.1 How I Worked With It

- **Planning.** It drafted the GitHub issues from the specification. I edited and created them, and decided the scope, for example that reporting needs an account and that email verification waits for Iteration 3.
- **Testing.** I had Claude Code write the automated tests for each feature as it was built, following conventions I set. I reviewed them and ran both suites before every pull request (Section 2).
- **Solving problems.** When something did not work, I described the problem and Claude Code helped me find the cause, by querying the data, reading the logs or taking screenshots of the app on the emulator. It suggested fixes, and I tested each one before accepting it (Section 8.2).

8.2 Solving Problems

Most of the problems in Table 3 were found and solved together with Claude Code, including the checks on the register's data, the location prompt and the "Near me" bug. It also helped with how the routes are drawn:

- **Line drawing on map.** Claude Code suggested two algorithms built into PostGIS for drawing the routes. Douglas-Peucker line simplification [10] removes points that barely change a route's shape, so all 28 routes load in about 78 KB. DBSCAN clustering [11] groups trailheads within 10 m of each other into sites. At 10 m, the Slieve Bloom Way, which starts 57 m from the Glenbarrow loops, stays a separate site.

8.3 Checking Its Work

Claude Code makes mistakes, so I check its claims against git, the tests and the running app. It also let me deliver more in Iteration 1 than planned, with more tests. Therefore, I must understand what is built, which is why I review every change.

9. Algorithms and Technical References

Table 5 lists the algorithms and standards TrailTrack relies on, and where each is used.

Algorithm or standard	Where it is used	Reference
Douglas-Peucker line simplification	ST_Simplify reduces each route for the map (tolerance 0.00005 degrees), about 78 KB for all 28 trails.	[10, 12]
DBSCAN clustering	ST_ClusterDBSCAN groups trailheads within 10 m of each other into sites, measured in metres in Irish Transverse Mercator.	[11]
Geodesic distance on the WGS 84 spheroid	"Near me" finds trails within a radius and sorts them by distance (ST_DWithin and ST_Distance on geography).	[13, 12]
R-tree spatial index on GiST	The index on trail geometry that keeps the distance search fast.	[14, 15]
bcrypt password hashing	Passwords are stored as bcrypt hashes. Passwords over 72 bytes, bcrypt's limit, are rejected.	[16]
Web standards	Login tokens are JSON Web Tokens signed with HMAC SHA-256. Each report has a UUID version 4 reference, so a report sent twice is stored once. Errors use Problem Details, and trail geometry is sent as GeoJSON.	[17, 18, 19, 20, 21, 22]

Table 5: Algorithms and standards used in v0.1

10. References and External Sources

[1] Sport Ireland (2026) *National Trails Register: GetIrelandActive trail routes* [ArcGIS feature service]. Available at: https://services-eu1.arcgis.com/CltcWyRoZmdwaB7T/ArcGIS/rest/services/GetIrelandActiveTrailRoutes/FeatureServer/0 (Accessed: 5 October 2026).

[2] Sport Ireland (2026) *National Trails Register: GetIrelandActive trail heads* [ArcGIS feature service]. Available at: https://services-eu1.arcgis.com/CltcWyRoZmdwaB7T/ArcGIS/rest/services/GetIrelandActiveTrailHeads/FeatureServer/0 (Accessed: 8 October 2026).

[3] OpenStreetMap contributors (2026) *OpenStreetMap*. Available at: https://www.openstreetmap.org/copyright (Accessed: 8 October 2026).

[4] Creative Commons (2013) *Attribution 4.0 International (CC BY 4.0)*. Available at: https://creativecommons.org/licenses/by/4.0/ (Accessed: 5 October 2026).

[5] Open Data Commons (2009) *Open Database License (ODbL) v1.0*. Available at: https://opendatacommons.org/licenses/odbl/ (Accessed: 8 October 2026).

[6] Esri (2026) *Query (Feature Service/Layer): ArcGIS REST APIs*. Available at: https://developers.arcgis.com/rest/services-reference/enterprise/query-feature-service-layer/ (Accessed: 5 October 2026).

[7] Coillte (2025) *Felling licence submitted to DAFM* [ArcGIS feature service]. Available at: https://services7.arcgis.com/6LSAdqgf43oKRpP6/arcgis/rest/services/COILLTE_FELLING_LICENCES/FeatureServer/0 (Accessed: 5 October 2026).

[8] Coillte (2026) *Trail maps: Oughaval Woods, Glenbarrow and Monicknew Woods*. Available at: https://www.coillte.ie/ (Accessed: 8 October 2026).

[9] Anthropic (2026) *Claude Code* [Computer program]. Available at: https://claude.com/claude-code (Accessed: 29 October 2026).

[10] Douglas, D.H. and Peucker, T.K. (1973) 'Algorithms for the reduction of the number of points required to represent a digitized line or its caricature', *The Canadian Cartographer*, 10(2), pp. 112-122.

[11] Ester, M., Kriegel, H.-P., Sander, J. and Xu, X. (1996) 'A density-based algorithm for discovering clusters in large spatial databases with noise', in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*. Portland, OR: AAAI Press, pp. 226-231.

[12] PostGIS Project (2026) *PostGIS 3.6 manual*. Available at: https://postgis.net/docs/ (Accessed: 8 October 2026).

[13] Karney, C.F.F. (2013) 'Algorithms for geodesics', *Journal of Geodesy*, 87(1), pp. 43-55.

[14] Guttman, A. (1984) 'R-trees: a dynamic index structure for spatial searching', in *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*. New York: ACM, pp. 47-57.

[15] Hellerstein, J.M., Naughton, J.F. and Pfeffer, A. (1995) 'Generalized search trees for database systems', in *Proceedings of the 21st International Conference on Very Large Data Bases*. Zurich, pp. 562-573.

[16] Provos, N. and Mazières, D. (1999) 'A future-adaptable password scheme', in *Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track*. Monterey, CA: USENIX Association, pp. 81-91.

[17] Jones, M., Bradley, J. and Sakimura, N. (2015) *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force. Available at: https://doi.org/10.17487/RFC7519 (Accessed: 10 October 2026).

[18] Jones, M. (2015) *JSON Web Algorithms (JWA)*. RFC 7518. Internet Engineering Task Force. Available at: https://doi.org/10.17487/RFC7518 (Accessed: 10 October 2026).

[19] Krawczyk, H., Bellare, M. and Canetti, R. (1997) *HMAC: keyed-hashing for message authentication*. RFC 2104. Internet Engineering Task Force. Available at: https://doi.org/10.17487/RFC2104 (Accessed: 10 October 2026).

[20] Davis, K., Peabody, B. and Leach, P. (2024) *Universally Unique IDentifiers (UUIDs)*. RFC 9562. Internet Engineering Task Force. Available at: https://doi.org/10.17487/RFC9562 (Accessed: 23 October 2026).

[21] Nottingham, M., Wilde, E. and Dalal, S. (2023) *Problem details for HTTP APIs*. RFC 9457. Internet Engineering Task Force. Available at: https://doi.org/10.17487/RFC9457 (Accessed: 10 October 2026).

[22] Butler, H., Daly, M., Doyle, A., Gillies, S., Hagen, S. and Schaub, T. (2016) *The GeoJSON format*. RFC 7946. Internet Engineering Task Force. Available at: https://doi.org/10.17487/RFC7946 (Accessed: 8 October 2026).